

Comandos de Git

1 - `git init`

- Inicializa un nuevo repositorio Git en la carpeta actual.
 - Crea la carpeta oculta `.git` donde se guarda toda la información del control de versiones.
-

2 - Ramas en Git

- Una rama es una línea de desarrollo independiente.
 - `master` o `main` suele ser la rama principal.
 - Se usan para trabajar en nuevas funcionalidades sin afectar el código estable.
-

3 - `git add` y `git commit`

- `git add <archivo>` → prepara cambios para ser guardados (stage).
 - `git commit -m "mensaje"` → guarda los cambios en el historial del repositorio.
-

4 - `git log` y `git status`

- `git log` → muestra el historial de commits.
 - `git status` → indica el estado actual de los archivos (modificados, preparados, sin seguimiento).
-

5 - `git checkout` y `git reset`

- `git checkout <rama|commit>` → moverte a otra rama o commit.
 - `git reset <commit>` → deshace cambios, moviendo HEAD a un commit anterior.
 - `--soft` → mantiene cambios en staging.
 - `--hard` → borra cambios y regresa al commit exacto.
-

6 - `git alias`

- Permite crear atajos para comandos largos.
- Ejemplo:

```
git config --global alias.lg "log --oneline --graph --decorate"
```

7 - Fichero `.gitignore`

- Archivo especial donde defines qué ficheros o carpetas no deben ser rastreados por Git.
- Útil para excluir archivos temporales, configuraciones locales o dependencias externas.

- Ejemplo típico:

```
node_modules/  
*.log  
.env
```

8 - `git diff`

- Muestra las diferencias entre el estado actual y el último commit.
- Permite ver qué líneas fueron añadidas, eliminadas o modificadas.
- Ejemplo:

```
git diff
```

9 - Desplazamiento en una rama

- Usar `git checkout <rama>` para moverte entre ramas.
- Permite trabajar en diferentes versiones del proyecto sin mezclar cambios.
- Ejemplo:

```
git checkout develop
```

10 - `git reset --hard` y `git reflog`

- `git reset --hard <commit>` → regresa la rama al commit indicado, borrando cambios en staging y en el directorio de trabajo.
- `git reflog` → muestra el historial de movimientos de HEAD (incluye cambios de rama, resets y commits).
- Ejemplo:

```
git reset --hard abc123  
git reflog
```

11 - `git tag`

- Sirve para marcar commits importantes, como versiones de lanzamiento.
- Se pueden crear tags ligeros o anotados.
- Ejemplo:

```
git tag v1.0.0
git push origin v1.0.0
```

12 - `git branch` y `git switch`

- `git branch` → lista, crea o elimina ramas.
 - `git switch <rama>` → cambia de rama (alternativa moderna a `git checkout`).
-

13 - `git merge`

- Fusiona los cambios de una rama en otra.
 - Puede generar un commit de merge si no es fast-forward.
-

14 - `git rebase`

- Reaplica commits de una rama sobre la punta de otra.
 - Reescribe el historial, evitando commits de merge.
-

15 - `git merge` vs `git rebase`

- `merge` conserva el historial completo con bifurcaciones.
 - `rebase` linealiza el historial, ideal para ramas de características.
-

16 - Fast-forward vs 3-way merge

- **Fast-forward:** la rama destino no tuvo nuevos commits; solo avanza la punta.
 - **3-way merge:** cuando ambas ramas tienen cambios distintos, Git crea un commit de merge.
-

17 - `git branch -d` y `git branch -D`

- `git branch -d <rama>` → elimina una rama ya fusionada.
 - `git branch -D <rama>` → fuerza la eliminación aunque no esté fusionada.
-

18 - `git log --oneline --graph --all`

- Muestra el historial de forma compacta y visual.
 - `--graph` dibuja el árbol de ramas.
 - `--all` muestra todas las ramas.
-

19 - Resolución de conflictos en Git

- Ocurre cuando dos ramas modifican la misma línea de un archivo.

- Git marca el archivo en conflicto y debes editarlo manualmente.
 - Luego usas `git add` y `git commit` para finalizar la fusión.
-

20 - `git stash`

- Guarda temporalmente los cambios sin commitear.
 - `git stash pop` → recupera los cambios guardados.
 - Útil para cambiar de rama sin perder trabajo en curso.
-

21 - Reintegración de ramas en Git

- Una vez terminada una funcionalidad, se integra a la rama principal.
 - Puede hacerse con `merge` o `rebase`.
-

22 - Eliminación de ramas en Git

- Después de fusionar, se eliminan las ramas para mantener limpio el repositorio.
 - `git branch -d <rama>` para ramas ya fusionadas.
-

23 - Introducción a GitHub

- Plataforma en la nube para alojar repositorios Git.
 - Facilita la colaboración, el código abierto y el trabajo en equipo.
-

24 - Primeros pasos en GitHub

- Crear una cuenta en github.com.
 - Crear un repositorio desde la interfaz web.
 - Conectarlo con tu repositorio local.
-

25 - Repositorio personal

- Es tu espacio en GitHub para almacenar proyectos.
 - Puede ser público o privado.
-

26 - Local y Remoto

- **Local:** repositorio en tu máquina.
 - **Remoto:** repositorio alojado en GitHub u otro servidor.
-

27 - Autenticación SSH en GitHub

- Permite conectar tu máquina con GitHub sin contraseña.

- Se genera una clave SSH con `ssh-keygen` y se agrega a GitHub.
-

28 - Repositorio proyecto

- Repositorio dedicado a un proyecto específico.
 - Incluye código, documentación y configuración.
-

29 - `git remote`

- `git remote add <nombre> <url>` → vincula un repositorio remoto.
 - `git remote -v` → lista los remotos configurados.
-

30 - Subida de un proyecto a GitHub

- `git push -u origin main` → sube la rama local al remoto.
 - La primera vez se usa `-u` para establecer seguimiento.
-

31 - `git fetch` y `git pull`

- `git fetch` → descarga cambios del remoto sin fusionarlos.
 - `git pull` → descarga y fusiona los cambios automáticamente (`fetch` + `merge`).
-

32 - `git clone`

- Clona un repositorio remoto en tu máquina local.
 - `git clone <url>` → crea una copia exacta del repositorio.
-

33 - `git push`

- Sube los commits locales al repositorio remoto.
 - `git push origin <rama>` → envía cambios a la rama indicada.
-

34 - `Fork` en GitHub

- Copia un repositorio ajeno a tu cuenta de GitHub.
 - Permite contribuir sin afectar el repositorio original.
-

35 - Flujo colaborativo en GitHub

- Hacer fork, clonar, crear rama, hacer cambios, PR.
 - El dueño del repo revisa y fusiona los cambios.
-

36 - Pull Request (PR) en GitHub

- Solicitud para fusionar tus cambios en el repositorio original.
 - Permite revisar código, discutir cambios y aprobar antes de fusionar.
-

37 - Ejercicio práctico

- Aplica los conceptos aprendidos en un flujo real.
 - Incluye fork, clonar, rama, commit, push y PR.
-

38 - Resolución de conflictos en Pull Requests

- Al hacer un PR, puede haber conflictos si el mismo archivo cambió.
 - Se resuelven igual que los conflictos locales, con `git merge` o desde la UI de GitHub.
-

39 - Sincronización de un Fork en GitHub

- Con el tiempo el repositorio original avanza.
 - Desde GitHub puedes hacer `sync fork` para actualizar tu copia.
-

40 - Markdown en GitHub

- Lenguaje de marcado para documentar repositorios.
 - Se usa en README.md, issues, PRs y wikis.
-

41 - Herramientas gráficas (GUI) para Git y GitHub

- GitHub Desktop, GitKraken, Sourcetree, VS Code.
 - Facilitan el uso de Git sin línea de comandos.
-

42 - Git y GitHub Flow

- Flujo de trabajo basado en ramas.
 - `main` siempre estable, ramas para características, PR para fusionar.
-

43 - Ejemplo Gitflow

- Gitflow usa `main`, `develop`, `feature/*`, `release/*` y `hotfix/*`.
 - Ideal para proyectos con ciclos de lanzamiento definidos.
-

44 - `git cherry-pick` y `git rebase`

- `git cherry-pick <commit>` → aplica un commit específico en otra rama.

- `git rebase -i` → reordenar, combinar o editar commits interactivamente.
-

45 - GitHub Pages y Actions

- **GitHub Pages:** aloja sitios web estáticos desde un repositorio.
- **GitHub Actions:** automatiza builds, tests y despliegues con CI/CD.